

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Yazılım Geliştirme I

Operatör Yükleme

Hafta 5

Operatör Yükleme

Doç. Dr. Tolga Berber

Bilgisayar Bilimleri · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Hafta 1–2'de iki noktayı toplamak için serbest bir fonksiyon çağırmak zorundaydınız; işlem çoğaldıkça ifade matematiksel gösterimden uzaklaşır:

```
// operatör yükleme YOK -- ic ice cagri yigini:
Vektor2B s = topla(carp(a, 3), b);
// operatör yükleme VAR -- dilin kendi sozdizimi:
Vektor2B s = a * 3 + b;
```

- Onlarca vektör işleminde `topla(...)`/`carp(...)` yığını okunmaz hale gelir
- Bu hafta "kendi tipiniz neden + ile toplanamıyor?" boşluğunu kapatacaksınız

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. Bir tipin yerleşik operatörlerle neden/nasıl kullanıldığını açıklamak; üye ile serbest (`friend`) operatörü ayırt etmek
2. Aritmetik (+, -, *) ve bileşik atama (+=) operatörlerini const doğruluğuyla yüklemek
3. C++20 üç yönlü karşılaştırmayı (<=>) `=default` ile altı operatöre türetmek
4. Akış (<</>>) ve indeksleme (`[]`) operatörlerini yazmak
5. Uçtan uca doğal davranan bir değer tipi (`Vektor2B`) tasarlamak

1 • Ön-öğretim Geçen Haftadan Bugüne

- Hafta 1–2'de `geometri::Nokta` için toplamayı bir serbest fonksiyonla yazmıştınız:

```
1 geometri::Nokta c = topla(a, b); // okunur ama matematige yabanci
```

- Oysa aklınızdaki matematiksel gösterim şuydu: `c = a + b`
- Bu hafta aynı `geometri` ad uzayına kız kardeş bir tip (`Vektor2B`) ekleyip dilin kendi sözdizimiyle yazacaksınız

BÖLÜM 1

Ünite 1 — Neden Operatör Yükleme?

2 • İçerik Operatör Yükleme Nedir?

Tanım: Operatör yükleme

Dilin yerleşik bir sembolünü (+, ==, << …) kendi tipiniz için `operator+` gibi özel isimli bir fonksiyon yazarak anlamlandırmaktır.

- Derleyici `a + b`'yi `a.operator+(b)` (üye) veya `operator+(a, b)` (serbest) çağrısına çevirir
- Sözdizimsel şeker; ama okunabilirliği ciddi biçimde artırır

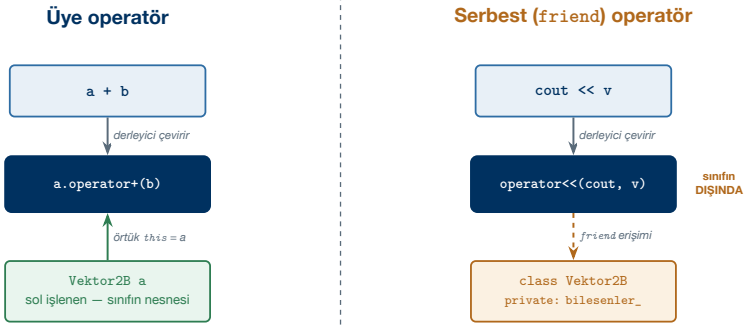
Analoji: Aynı Tuş, Cihaza Göre Anlam

Analoji

Bir uzaktan kumandadaki “+” tuşu televizyonda sesi, klimada sıcaklığı artırır — sembol aynıdır, anlamı cihaza göre değişir. C++’ta da + sembolü `int` için sayısal toplama, `Vektor2B` için vektör toplamı anlamına gelir.

- Sembolü siz tipinize göre anlamlandırırsınız
- Ama anlamı sezgiye sadık tutmak sizin sorumluluğunuzdadır

Üye mi, Serbest mi?



Sol işlenen sizin tipinizse üye olabilir; değilse serbest/`friend`.

İlk Operatör: Üye `operator+`

`Vektor2B`, `geometri` ad uzayında bir `std::array<int,2>` tutar; ilk somut operatörü üye `operator+`:

```
42    std::array<int, 2> bilesenler_;
```

```
14 Vektor2B Vektor2B::operator+(const Vektor2B& sag) const {
15    return Vektor2B(bilesenler_[0] + sag.bilesenler_[0],
16                    bilesenler_[1] + sag.bilesenler_[1]);
17 }
```

- Sol işlenen örtük `this`; sağ işlenen `const` referans

`friend`: Güvenilen Kişiye Anahtar

Analoji

Normalde eve (private alanlara) yalnızca ev sahibi (üye fonksiyonlar) girer. Ev sahibi, güvendiği belirli bir kişiye (`friend` fonksiyona) özel olarak anahtar verebilir — herkese değil, yalnız isim isim belirtilene.

- Yalnız public arayüzü kullanan serbest fonksiyon `friend` *gerektirmez*
- private alana doğrudan erişmesi gerekirse `friend` bildirilir (Ünite 4'te somutlaşacak)

BÖLÜM 2

Ünite 2 — Aritmetik ve Bileşik Atama

2 • İçerik Aritmetik Bildirimleri

```
19  Vektor2B operator+(const Vektor2B& sag) const;
20  Vektor2B operator-(const Vektor2B& sag) const;
21  Vektor2B operator*(int katsayi) const;
22  Vektor2B& operator+=(const Vektor2B& sag);
```

- `operator+`, `operator-`, `operator*` salt okunur → const
- `operator+=` nesneyi *yerinde* değiştirir → const değil

Simetri Sorunu: v*3 çalışır, 3*v çalışmaz

```
47 Vektor2B operator*(int katsayi, const Vektor2B& v);
```

```
42 // friend DEGILDIR: yalniz Vektor2B'nin public operator*'ina devreder
       .
43 Vektor2B operator*(int katsayi, const Vektor2B& v) {
44    return v * katsayi;
45 }
```

- `v * 3` = `v.operator*(3)` çalışır; ama `int`'in üye operatörü olamayacağı için `3 * v` serbest ister
- Yalnız public `operator*`'a devreder — friend değildir

Bileşik Atama: Yerinde Güncelleme

Analoji

`+=` bir biriktirme kumbarası gibidir: her katkıda yeni bir kumbara almaz (yeni nesne oluşturmaz), var olanın üstüne eklersiniz — nesne yerinde güncellenir.

```
34 Vektor2B& Vektor2B::operator+=(const Vektor2B& sag) {
35     bilesenler_[0] += sag.bilesenler_[0];
36     bilesenler_[1] += sag.bilesenler_[1];
37     return *this;
38 }
```

- `*this`'i (kendi nesnesini) referansla döndürür → zincirlenebilir

Neden Değere Döndürme? (Tuzak)

Ortak Hata / Tuzak

`operator+` sonucu ikisinden de bağımsız yeni bir nesnedir; değer (by value) döndürülür. Yerel bir değişkene *referans* döndürmek, fonksiyon bitince sarkan (dangling) bir referans üretir.

Kesir köprüsü (kod yok)

Değer tipiniz bir `Kesir` olsaydı, her işlemden sonra sonucu sadeleştirmek (gcd) gerekirdi — operatör içinde tutarlı/kanonik bir sonuç üretmek sizin sorumluluğunuzdadır.

BÖLÜM 3

Ünite 3 — Karşılaştırma ve <=>

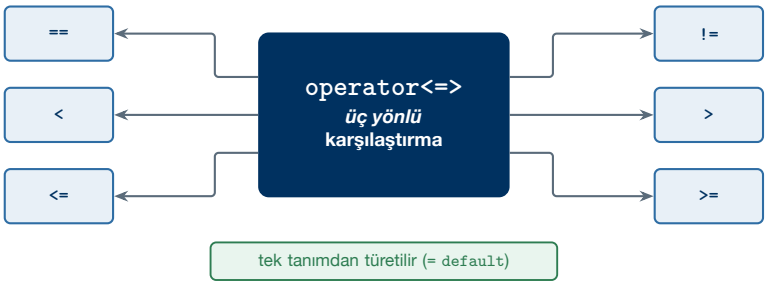
2 • İçerik Altı Operatörün Yükü

- C++17 ve öncesinde eşitlik ve sırayı tanımlamak için altı ayrı operatör (==, !=, <, <=, >, >=) elle yazılırdı
- Hepsi aynı temel soruyu sorar: “sol mu küçük, eşit mi, büyük mü?”
- Tekrar çok, tutarsızlık riski yüksek

Analoji

Bir terazi tek ölçümde “sol mu ağır, eşit mi, sağ mı ağır” der; bu tek ölçümden altı sorunun tümü otomatik cevaplanır.

Spaceship: Tek Tanımdan Altı Operatör



C++20 `operator<=>`: tek tanımdan derleyici altı karşılaştırma operatörünü türetir.

= `default` ile Karşılaştırma

```
26  bool operator==(const Vektor2B&) const = default;
27  auto operator<=>(const Vektor2B&) const = default;
```

- `bilesenler_` üye sırasına göre sözlüksel karşılaştırılır: önce 0., eşitse 1. bileşen
- `<=>` yanında `==`'i ayrıca `default`'lamak yaygın kalıp: salt eşitlik genelde daha ucuzdur
- Tam sayı alanlar → `std::strong_ordering` (kayan nokta olsaydı `partial_ordering`)

Ne Zaman Elle <=>? (Dikkat)

= default'ın sınırı

= default, yalnızca üyelerin bildirim sırasındaki karşılaştırması anlamsal sıraya eşitse doğrudur.

- Bir Kesir(pay, payda) için sözlüksel sıra sayısal sırayla örtüşmez: 1/2 ile 1/3'te sözlükçe 1/2 < 1/3 çıkar ama sayıca 1/2 > 1/3
- Böyle durumlarda <=> elle, çapraz çarpımla yazılır
- Vektor2B'de sözlüksel sıra pratikte yeterli (std::sort/std::set için tutarlı toplam sıralama)

BÖLÜM 4

Ünite 4 — Akış (<</>>) ve [ ]

2 • İçerik İndeksleme: const/non-const Çifti

`[]` apartman dairesi numarası gibidir: bileşene adı yerine numarasıyla ulaşırsınız (`v[0]`, `v[1]`).

```
49 int& Vektor2B::operator[](int indeks) {
50     // On kosul: indeks yalniz 0 ya da 1 olmalidir (hata yonetimi
           Hafta 13).
51     return bilesenler_[static_cast<std::size_t>(indeks)];
52 }
53
54 int Vektor2B::operator[](int indeks) const {
55     return bilesenler_[static_cast<std::size_t>(indeks)];
56 }
```

- non-const sürüm `int&` döndürür → `v[0] = 10;`
- const sürüm salt okunur `int` döndürür (geçerli indeks: 0/1)

Akış Operatörleri: Gerçek `friend`

Analoji

`<<` değeri akışın içine "akıtır" (yazdırır), `>>` akıştan "dışarı çeker" (okur) — ok akışın yönünü gösterir.

```
36  friend std::ostream& operator<<(std::ostream& cikis, const
        Vektor2B& v);
37  friend std::istream& operator>>(std::istream& giris, Vektor2B& v);
```

- Sol işlenen `std::ostream`/`std::istream` → üye olamaz, serbest zorunlu
- `private` `bilesenler_`'e erişmek için friend

Akış Gövdeleri ve Zincirleme

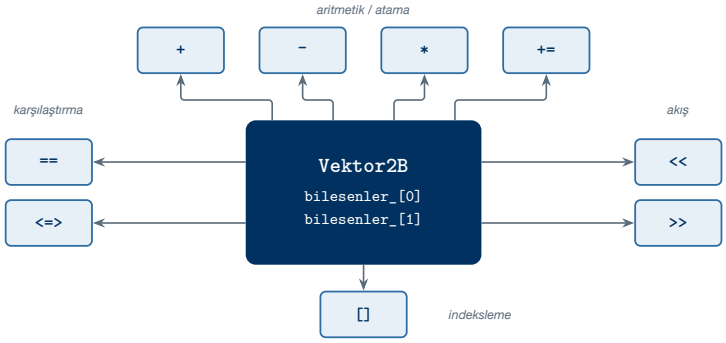
```
60 std::ostream& operator<<(std::ostream& cikis, const Vektor2B& v) {
61    cikis << '(' << v.bilesenler_[0] << ", " << v.bilesenler_[1] << ')
          ';
62    return cikis;
63 }
```

- Akışı referansla geri döndürmek zincirlemeyi sağlar: `std::cout << v1 << v2;`
- `operator>>` için demoda `std::istringstream` ile sabit metinden okunur (deterministik)

Uçtan Uca: Kullanım

```
13    Vektor2B v1(2, 3);
14    Vektor2B v2(4, 1);
15    std::cout << "v1=" << v1 << ", v2=" << v2 << '\n';
16    std::cout << "v1+v2=" << (v1 + v2) << '\n';
17    std::cout << "v1-v2=" << (v1 - v2) << '\n';
18    std::cout << "v1*3=" << (v1 * 3) << '\n';
19    std::cout << "3*v1=" << (3 * v1) << '\n';
20    v1 += v2;
21    std::cout << "v1 += v2 sonrasi v1=" << v1 << '\n';
```

Tamamlanmış Bir Değer Tipi



Artık `+ - * += == <=> << >> []` ile `int` gibi doğal davranır.

BÖLÜM 5

Özet

Özet

Özet

- Operatör yükleme: yerleşik sembolü `operator+` gibi bir fonksiyonla kendi tipinize anlamlandırma
- Sol işlenen sizin tipinizse üye; değilse serbest, private erişimde `friend`
- Aritmetik değere döner ve `const`'tır; `+=` yerinde günceller, `*this`'i referansla döner
- C++20 `<=> = default`: tek tanımdan altı operatör (anlamsal sıra uyuşmazsa elle)
- `<</>>` serbest+friend; `[]` const/non-const çift

5 • İzleme Aktarım — Başka Nerede?

Aynı "üye/serbest/`friend` + değere/referansa döndürme" kalıbı her değer tipinde geçerlidir; `Vektor2B`'ye özgü değildir:

- Bir `Kesir` veya `Para` tipinde `operator+` yeni bir sonuç *oluşturup* değerle döner; `+=` yerinde günceller — Kitaplık'ta bir gecikme günü birikimi de `+=` ile modellenir
- `<=> = default`'un "üye sırası = anlamsal sıra" varsayımı her tipte sorgulanır: `Kitap`'ı başlığa/yıla göre sıralarken `<=>`'i elle yazmanız gerekebilir
- Bu "değer tipini operatörlerle tamamlama" becerisi Hafta 6'da `Materyal` → `BasiliKitap`/`EKitap` hiyerarşisi kurulurken doğrudan yeniden kullanılır

Sırada Ne Var?

Bu hafta lab'da

`Vektor2B`'nin operatörlerini *kendi elinizle* tamamlayacak; aritmetik, karşılaştırma, akış ve indeksleme demolarını derleyip çalıştıracaksınız.

- Kendinizi Sınayın her üniteyi kapsıyor: `operator<<` neden üye olamaz, `operator+` neden değere döner, `= default` ne zaman yetmez, `[]` neden const/non-const çift
- Gelecek hafta: birden çok ilişkili tipi (`Materyal` → `BasiliKitap`/`EKitap`) kalıtım & kompozisyonla organize etmek